GURU GHASIDAS VISHWAVIDYALAYA

(Central University) Bilaspur, Chhattisgarh

Assignment

(2025-2026)

TOPIC :- OOPs Concept based Programs

Name:- *Nitin Babu*

Roll no:- *24070142*

Dept.:- *(BSc CS) CSIT*

Subject:- *OOPs using JAVA (Major 1)*

Submitted to :

Mr. Ratnesh Srivastava

1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

```
//Code
```

```
1  import java.io.*;
 2 import java.util.*;
 3
 4 class Farmer {
 5     String name, fruit, city;
 6     int quantity;
 7
 8     Farmer(String name, String fruit, String city, int quantity) {
 9         this.name = name;
10         this.fruit = fruit;
11         this.city = city;
12         this.quantity = quantity;
13    }
14 }
15 public class FarmerData {
16    public static void main(String[] args) throws IOException {
17        Scanner sc = new Scanner(System.in);
18        ArrayList<Farmer> list = new ArrayList<>();
19
20      // Input
21      for (int i = 0; i < 10; i++) {
22           System.out.println("Enter Farmer Name, Fruit, City,
23 Quantity:");
24           String name = sc.next();
25           String fruit = sc.next();
26           String city = sc.next();
27           int quantity = sc.nextInt();
28
29           list.add(new Farmer(name, fruit, city, quantity));
30       }
31       // Write BEFORE sorting
32       BufferedWriter bw = new BufferedWriter(new
33 FileWriter("data.txt"));
34       bw.write("----- BEFORE SORTING -----\n");
35       bw.write("Name\tFruit\tCity\tQuantity\n");
36
37       for (Farmer f : list) {
38            bw.write(f.name + "\t" + f.fruit + "\t" + f.city + "\t" +
39 f.quantity +
40 "\n");
41        }
42 // Sorting by quantity
43
44         list.sort((a, b) -> a.quantity - b.quantity);
45         // Write AFTER sorting
```

```
        bw.write("\n----- AFTER SORTING -----\n");
        bw.write("Name\tFruit\tCity\tQuantity\n");

        for (Farmer f : list) {
            bw.write(f.name + "\t" + f.fruit + "\t" + f.city + "\t" +
f.quantity +
"\n");
        }

        bw.close();

        System.out.println("Data saved to file (data.txt)");

        // Search
        System.out.print("Enter farmer name to search: ");
        String searchName = sc.next();

        boolean found = false;

        for (Farmer f : list) {
            if (f.name.equalsIgnoreCase(searchName)) {
                System.out.println("Found:");
                System.out.println("Name: " + f.name);
                System.out.println("Fruit:
" + f.fruit);
                System.out.println("City: " + f.city);
                System.out.println("Quantity: " + f.quantity);
                found = true;
            }
        }

        if (!found) {
            System.out.println("Farmer not found.");
        }

        sc.close();
    }
```

```
//Output
```

```
Enter Farmer Name, Fruit, City, Quantity:
 Ram Mango Delhi 500
 Enter Farmer Name, Fruit, City, Quantity:
 Shyam Apple Mumbai 1200
 Enter Farmer Name, Fruit, City, Quantity:
 Amit Banana Pune 300
 Enter Farmer Name, Fruit, City, Quantity:
 Ravi Orange Delhi 800
 Enter Farmer Name, Fruit, City, Quantity:
 Sita Mango Jaipur 1500
 Enter Farmer Name, Fruit, City, Quantity:
 Geeta Apple Indore 200
```

```
Enter Farmer Name, Fruit, City, Quantity:
Mohan Banana Bhopal 900
Enter Farmer Name, Fruit, City, Quantity:
Ramesh Mango Delhi 700
Enter Farmer Name, Fruit, City, Quantity:
Kiran Apple Pune 400
Enter Farmer Name, Fruit, City, Quantity:
Neha Orange Mumbai 600
Data saved to file (data.txt)
Enter farmer name to search: kiran
Found:
Name: Kiran
Fruit: Apple
City: Pune
Quantity: 40
```

2.Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

```
//Code
```

```java
import java.util.*;

class Fruit {
   String name;
   int quantity;
   String quality;

  Fruit(String name, int quantity, String quality) {
       this.name = name;
       this.quality = quality;

       // If quantity > 1000, store half
       if (quantity > 1000) {
           this.quantity = quantity / 2;
       } else {
           this.quantity = quantity;
       }
   }

   void display() {
       System.out.println("Fruit: " + name +
               ", Quantity: " + quantity +
               ", Quality: " + quality);
   }
}

ublic class FruitProgram {
   public static void main(String[] args) {
```

```java
        Scanner sc = new Scanner(System.in);

        Fruit[] fruits = new Fruit[3];

        String[] fruitNames = { "Mango", "Apple", "Papaya" };

        // Input
        for (int i = 0; i < 3; i++) {
            System.out.println("Enter quantity for " + fruitNames[i] +
": ");
            int qty = sc.nextInt();

            System.out.println("Enter quality (good/bad/average): ");
            String quality = sc.next();

            fruits[i] = new Fruit(fruitNames[i], qty, quality);
        }

        // Display all data
        System.out.println("\nStored Data:");
        for (Fruit f : fruits) {
            f.display();
        }

        // Search
        System.out.println("\nEnter fruit name to search:");
        String searchFruit = sc.next();

        System.out.println("Enter quality to search:");
        String searchQuality = sc.next();

        boolean found = false;

        for (Fruit f : fruits) {
            if (f.name.equalsIgnoreCase(searchFruit) &&
                    f.quality.equalsIgnoreCase(searchQuality)) {

                System.out.println("\nFruit Found:");
                f.display();
                found = true;
            }
        }

        if (!found) {
            System.out.println("\nNo matching fruit found.");
        }

        sc.close();
    }
}
```

```
//Output
```

```
Enter quantity for Mango:
 1600
 Enter quality (good/bad/average):
 good
 Enter quantity for Apple:
 500
 Enter quality (good/bad/average):
 bad
 Enter quantity for Papaya:
 133
 Enter quality (good/bad/average):
 good

 Stored Data:
 Fruit: Mango, Quantity: 800, Quality: good
 Fruit: Apple, Quantity: 500, Quality: bad
 Fruit: Papaya, Quantity: 133, Quality: good

 Enter fruit name to search:
 papaya
 Enter quality to search:
 good

 Fruit Found:
 Fruit: Papaya, Quantity: 133, Quality: good
```

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.

```
//Code
```

```
import java.util.*;

class Orchard {
    String name;
    String fruit;
    String quality;

    Orchard(String name, String fruit, String quality) {
        this.name = name;
        this.fruit = fruit;
        this.quality = quality;
    }
}


public class OrchardTree {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```java

        // Predefined orchards
        Orchard[] orchards = {
                new Orchard("A", "Apple", "Good"),
                new Orchard("B", "Litchi", "Good"),
                new Orchard("C", "Mango", "Good"),
                new Orchard("D", "Apple", "Bad")
        };

        // Input for search
        System.out.print("Enter fruit to search: ");
        String searchFruit = sc.next();

        boolean found = false;

        System.out.println("\nTree View:\n");
        System.out.println("Orchards");

        for (Orchard o : orchards) {
            if (o.fruit.equalsIgnoreCase(searchFruit)) {
                System.out.println("├── " + o.name + " (" + o.fruit +
", " + o.quality
+ ")  <-- Found");
                found = true;
            } else {
                System.out.println("├── " + o.name + " (" + o.fruit +
", " + o.quality
+ ")");
            }
        }

        if (!found) {
            System.out.println("\nFruit not found in any orchard.");
        }

        sc.close();
    }
}
```

```
//Output
```

```
Enter fruit to search: litchi

 Tree View:
 Orchards
 â??â??â?? A (Apple, Good)
 â??â??â?? B (Litchi, Good)  <-- Found
 â??â??â?? C (Mango, Good)
 â??â??â?? D (Apple, Bad)
```

4. Take a map provided as a graph & display all the subtrees of connected points.

```
//Code
```

```java
import java.util.*;

public class GraphSubtrees {

    static void dfs(int node, boolean[] visited,
ArrayList<ArrayList<Integer>> graph,
List<Integer> component) {
        visited[node] = true;
        component.add(node);

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                dfs(neighbor, visited, graph, component);
            }
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        ArrayList<ArrayList<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++)
            graph.add(new ArrayList<>());

        // Input edges
        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            graph.get(u).add(v);
            graph.get(v).add(u); // undirected graph
        }

        boolean[] visited = new boolean[n];

        System.out.println("\nConnected Subtrees:");

        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                List<Integer> component = new ArrayList<>();
                dfs(i, visited, graph, component);

                System.out.println(component);
            }
        }

```

```
        sc.close();
    }
}
```

```
//Output
```

```
Enter number of vertices: 9
 Enter number of edges: 7
 Enter edges (u v):
 3 4
 1 3
 2 4
 5 6
 4 5
 3 6
 1 4

 Connected Subtrees:
 [0]
 [1, 3, 4, 2, 5, 6]
 [7]
 [8]
```

5.Enter entry point & goal point & produce the matching subtree of the graph.

```
///Code
```

```
import java.util.*;

public class GraphPath {

    static boolean dfs(int current, int goal, boolean[] visited,
            ArrayList<ArrayList<Integer>> graph,
            List<Integer> path) {

        visited[current] = true;
        path.add(current);

        if (current == goal)
            return true;

        for (int neighbor : graph.get(current)) {
            if (!visited[neighbor]) {
                if (dfs(neighbor, goal, visited, graph, path))
                    return true;
            }

        }

```

```java
        // Backtrack if path not found
        path.remove(path.size() - 1);
        return false;
    }

    public static void main(String[] rgs) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of vertices: ");
        int n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();

        ArrayList<ArrayList<Integer>> graph = new ArrayList<>();

        for (int i = 0; i < n; i++)
            graph.add(new ArrayList<>());

        System.out.println("Enter edges (u v):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();

            graph.get(u).add(v);
            graph.get(v).add(u);
        }

        System.out.print("Enter entry point: ");
        int start = sc.nextInt();

        System.out.print("Enter goal point: ");
        int goal = sc.nextInt();
        boolean[] visited = new boolean[n];
        List<Integer> path = new ArrayList<>();

        if (dfs(start, goal, visited, graph, path)) {
            System.out.println("Matching Subtree (Path): " + path);
        } else {
            System.out.println("No path exists between given points.");
        }

        sc.close();
    }
}
```

```
//Output
```

```
Enter number of vertices: 9
 Enter number of edges: 7
 Enter edges (u v):
 1 2
 1 3
 2 4
 4 5
 5 3
```

```
1 6
4 3
Enter entry point: 1
Enter goal point: 4
Matching Subtree (Path): [1, 2, 4]
```

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

```
//Code
```

```java
import java.util.*;

public class ShortestPath {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n, e;
        System.out.print("Enter number of vertices: ");
        n = sc.nextInt();

        System.out.print("Enter number of edges: ");
        e = sc.nextInt();

        int[][] graph = new int[n][n];

        // Initialize with large value
        for (int i = 0; i < n; i++)
           Arrays.fill(graph[i], 9999);

        System.out.println("Enter edges (u v weight):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            int w = sc.nextInt();

            graph[u][v] = w;
            graph[v][u] = w;
        }

        System.out.print("Enter start node: ");
        int start = sc.nextInt();

        System.out.print("Enter end node: ");
        int end = sc.nextInt();

        int[] dist = new int[n];
        boolean[] visited = new boolean[n];
        int[] parent = new int[n];

        Arrays.fill(dist, 9999);
```

```java
        Arrays.fill(parent, -1);

        dist[start] = 0;

        // Simple Dijkstra (without priority queue)
        for (int i = 0; i < n - 1; i++) {

            int min = 9999, u = -1;

            for (int j = 0; j < n; j++) {
                if (!visited[j] && dist[j] < min) {
                    min = dist[j];
                    u = j;
                }
            }

            visited[u] = true;

            for (int v = 0; v < n; v++) {
                if (graph[u][v] != 9999 && !visited[v]) {
                    if (dist[u] + graph[u][v] < dist[v]) {
                        dist[v] = dist[u] + graph[u][v];
                        parent[v] = u;
                    }
                }
            }
        }

        // Print path
        List<Integer> path = new ArrayList<>();
        for (int v = end; v != -1; v = parent[v])
            path.add(v);

        Collections.reverse(path);

        System.out.println("Shortest Distance: " + dist[end]);
        System.out.println("Path: " + path);

        sc.close();
    }
}
```

```
//Output
```

```
Enter number of vertices: 5
 Enter number of edges: 4
 Enter edges (u v weight):
 1 3 2
 1 4 2
 1 2 3
 1 3 4
 Enter start node: 1
 Enter end node: 3
 Shortest Distance: 4
 Path: [1, 3]
```

7. Write a program to demonstrate the use of all keywords made for exception handling.

```
//Code
```

```java
import java.util.*;

public class ExceptionDemo {

    // using 'throws'
    static void checkNumber(int num) throws Exception {
        if (num < 0) {
            // using 'throw'
            throw new Exception("Number cannot be negative");
        } else {
            System.out.println("Valid number: " + num);
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try { // using 'try'

            System.out.print("Enter a number: ");
            int num = sc.nextInt();

            int result = 10 / num; // may cause ArithmeticException
            System.out.println("Result: " + result);

            checkNumber(num); // method that uses throws

        } catch (ArithmeticException e) { // using 'catch'
            System.out.println("Error: Cannot divide by zero");

        } catch (Exception e) { // general exception
            System.out.println("Exception: " + e.getMessage());


        } finally { // using 'finally'
            System.out.println("Program finished (finally block
executed)");
        }

        sc.close();
    }
}
```

```
//Output
```

```
Enter a number: -3
 Result: -3
 Exception: Number cannot be negative
 Program finished (finally block executed)
```

8. Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

```
//Code
```

```java
import java.util.*;

// Custom Exception
class NoMoneyException extends Exception {
   NoMoneyException(String message) {
       super(message);
   }
}

public class BankAccount {

   static void withdraw(int balance, int amount) throws
NoMoneyException {
       if (amount > balance) {
           throw new NoMoneyException("Insufficient Balance!");
       } else {
           balance -= amount;
           System.out.println("Withdrawal successful");
           System.out.println("Remaining Balance: " + balance);
       }
   }

   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);

       System.out.print("Enter account balance: ");
       int balance = sc.nextInt();

       System.out.print("Enter amount to withdraw: ");
       int amount = sc.nextInt();

       try {
           withdraw(balance, amount);
       } catch (NoMoneyException e) {
           System.out.println("Exception: " + e.getMessage());
       }

       sc.close();
   }
}
```

```
//Output
```

```
Enter account balance: 5000
 Enter amount to withdraw: 600
 Withdrawal successful
 Remaining Balance: 4400

 Enter account balance: 500
 Enter amount to withdraw: 1000
 Exception: Insufficient Balance!
```

9. Write a program which enables to use a platform by a train one at a time.

```
//Code
```

```java
class Platform {

    synchronized void usePlatform(String trainName) {
        System.out.println(trainName + " is entering the platform");

        try {
            Thread.sleep(2000); // simulate time on platform
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println(trainName + " is leaving the platform\n");
    }
}

class Train extends Thread {
    Platform p;
    String name;

    Train(Platform p, String name) {
        this.p = p;
        this.name = name;
    }

    public void run() {
        p.usePlatform(name);
    }
}

public class RailwayStation {
    public static void main(String[] args) {

        Platform platform = new Platform();

        Train t1 = new Train(platform, "Train A");
        Train t2 = new Train(platform, "Train B");
        Train t3 = new Train(platform, "Train C");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

```
//Output
```

```
Train A is entering the platform
 Train A is leaving the platform

 Train C is entering the platform
 Train C is leaving the platform

 Train B is entering the platform
 Train B is leaving the platform
```

10. Write a program of Restaurant and Waiter, which accept order if kitchen if free and chef deliver orders if he is free.

```
//Code
```

```java
class Restaurant {

    boolean orderReady = false;


    synchronized void placeOrder(String order) {
        try {
            while (orderReady) {
                wait(); // wait if kitchen is busy
            }

            System.out.println("Waiter placed order: " + order);
            orderReady = true;

            notify(); // notify chef

        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }

    // Chef preparing order
    synchronized void prepareOrder() {
        try {
            while (!orderReady) {
                wait(); // wait if no order
            }

            System.out.println("Chef is preparing order...");
            Thread.sleep(2000); // simulate cooking

            System.out.println("Chef delivered the order\n");
            orderReady = false;

            notify(); // notify waiter

        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
 }

 class Waiter extends Thread {
    Restaurant r;

    Waiter(Restaurant r) {
        this.r = r;
    }

    public void run() {
        String[] orders = { "Pie", "Rice", "Pasta" };
```

```java
        for (String order : orders) {
            r.placeOrder(order);
        }
    }
}

class Chef extends Thread {
    Restaurant r;

    Chef(Restaurant r) {
        this.r = r;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            r.prepareOrder();
        }
    }

}
public class RestaurantDemo {
    public static void main(String[] args) {

        Restaurant r = new Restaurant();

        Waiter w = new Waiter(r);
        Chef c = new Chef(r);
        w.start();
        c.start();
    }
}
```

```
//Output
```

```
Waiter placed order: Pie
Chef is preparing order...
Chef delivered the order

Waiter placed order: Rice
Chef is preparing order...
Chef delivered the order

Waiter placed order: Pasta
Chef is preparing order...
Chef delivered the order
```